

UNIVERSITÉ DE YAOUNDÉ I
THE UNIVERSITY OF YAOUNDE I
FACULTÉ DES SCIENCES
DÉPARTEMENT D'INFORMATIQUE

RAPPORT DE TRAVAIL PRATIQUE
INF3421 — Langage Formel & Compilation

Boîte à outils interactive pour la théorie des langages et des automates finis

Réalisé par :
KAMDOM NGUETCHESSI MERVEILLE PRISCA
Matricule : 23V2060
Licence 3 Informatique

Sous la supervision de :
Pr. Etienne Kouokam

Année académique 2025 – 2026

Sommaire

- 1. Introduction
- 2. Architecture générale du projet
- 3. Représentation d'un automate fini
- 4. Systèmes d'équations linéaires en langages (lemme d'Arden)
- 5. Constructions d'un automate à partir d'une expression régulière
- 6. Déterminisation, complétion, émondage
- 7. Minimisation et automate canonique
- 8. Opérations de clôture
- 9. États, reconnaissance, énumération, équivalence
- 10. Le moteur graphique PGM
- 11. Interface interactive
- 12. Validation et tests
- 13. Difficultés rencontrées et solutions apportées
- 14. Conclusion

1. Introduction

Le cours INF3421 (Langage Formel & Compilation) couvre l'ensemble de la théorie des langages formels nécessaire à la construction d'un compilateur : mots et langages, systèmes d'équations linéaires en langages et lemme d'Arden, langages réguliers et automates finis (déterministes, non-déterministes, avec ε -transitions), propriétés de clôture, minimisation et automate canonique, puis application à l'analyse lexicale.

L'objectif de ce TP est de transformer cette théorie en un outil logiciel opérationnel : une bibliothèque C écrite intégralement « from scratch » (sans bibliothèque externe, hormis la bibliothèque mathématique standard pour le tracé des cercles), exposant chaque algorithme du cours à travers un menu interactif convivial. Chaque opération affiche un résultat textuel détaillé (table de transition, traces de calcul) et génère automatiquement une image au format PGM représentant graphiquement l'automate obtenu, sauvegardée dans un dossier dédié.

Ce rapport documente les choix de conception, explique chaque algorithme implémenté en le reliant précisément à la section du cours dont il est issu, présente les résultats de validation, et revient sur les principales difficultés rencontrées durant le développement.

2. Architecture générale du projet

Le projet est organisé en quatre modules indépendants, chacun exposé par un fichier d'en-tête et implémenté dans un fichier source dédié, plus un point d'entrée regroupant le menu interactif :

Module	Rôle	Fichiers
pgm	Moteur graphique minimal : canevas, tracé de segments (Bresenham), de cercles (cercle médian), police bitmap 5×7, sauvegarde au format P5.	pgm.h / pgm.c
automaton	Structure de données Automaton (unique pour AFD/AFN/ ε -AFN) et tous les algorithmes portant sur les automates.	automaton.h / automaton.c
regex	Arbre syntaxique des expressions régulières, analyseur syntaxique descendant récursif, simplification algébrique.	regex.h / regex.c
equation	Résolution de systèmes d'équations linéaires en langages (lemme d'Arden / méthode de Gauss), constructions de Thompson et de Glushkov.	equation.h / equation.c
main	Menu interactif : saisie, enchaînement des opérations, affichage, sauvegarde des images.	main.c

Un choix de conception central est de représenter, dans une seule et même structure Automaton, les trois familles d'automates vues au chapitre 3 du cours : automate déterministe (définitions 1 et 4), automate non-déterministe (définition 6) et automate non-déterministe à transitions spontanées (définition 7). La structure comporte un drapeau hasEpsilon et une matrice de transition à trois dimensions `delta[q][symbole][q']` dans laquelle un indice de symbole réservé (EPS_IDX) code la transition spontanée. Cette unification évite toute duplication de code : les mêmes fonctions s'appliquent

indifféremment à un AFD, un AFN ou un ε -AFN, et une simple lecture du drapeau `hasEpsilon` et du nombre de transitions sortantes par état suffit à décider du degré de généralité de l'automate manipulé (cf. fonctions `aut_is_deterministic` et `aut_is_complete`).

```
typedef struct {
    int nStates;
    int nSymb;
    char alphabet[MAX_SYMB];
    int hasEpsilon;
    unsigned char delta[MAX_STATES][MAX_SYMB + 1][MAX_STATES];
    unsigned char isInitial[MAX_STATES];
    unsigned char isFinal[MAX_STATES];
    char name[MAX_STATES][NAME_LEN];
} Automaton;
```

Les ensembles d'états (utilisés massivement pour l' ε -fermeture et la construction des sous-ensembles) sont représentés par des entiers 64 bits (BitSet), chaque bit codant l'appartenance d'un état, ce qui rend les opérations ensemblistes (union, test d'appartenance, comparaison) immédiates et très rapides, sans allocation dynamique.

3. Représentation d'un automate fini

Le module `main.c` propose deux façons de définir l'automate courant `A` : la saisie manuelle complète (nombre d'états, alphabet, états initiaux/finaux, puis liste de transitions au format « origine symbole destination », le symbole '@' désignant une transition spontanée), ou le chargement de l'un des cinq automates d'exemple, chacun choisi pour illustrer un aspect précis du cours :

Exemple	Description	Illustre
1	AFN non-déterministe reconnaissant $\{a,b\}^*.ab$	Détermination (construction des sous-ensembles)
2	ε -AFN pour $a^*b^*c^*$ (cf. figure 3.11 du cours)	ε -fermeture, ε -AFN $\rightarrow$ AFN, ε -AFN $\rightarrow$ AFD
3	AFD comptant les « a » modulo 3 (cf. figure 3.3)	Automate déjà complet — vérification de la complétion
4	Automate possédant des états inaccessibles / non co-accessibles	États utiles et émondage
5	AFD redondant reconnaissant les mots finissant par « a »	Minimisation de Moore (fusion d'états équivalents)

Chaque affichage (fonction `aut_print`) présente les états, l'alphabet, les ensembles d'états initiaux et finaux, le statut « déterministe / complet », puis la table de transition complète. Cet affichage est systématiquement doublé d'une image PGM générée par `aut_render_pgm` (voir section 10).

4. Systèmes d'équations linéaires en langages (lemme d'Arden)

Le cours établit (section 2.3) que l'équation $X = A.X + B$ admet $A^*.B$ comme plus petite solution (lemme d'Arden), et que cette résolution s'étend à un système de n équations linéaires par la « méthode de Gauss » : résolution partielle d'une inconnue (élimination de son propre coefficient par le lemme d'Arden), puis substitution du résultat dans les autres équations. Le cours souligne aussi (section 3.3.2) que l'extraction de l'expression régulière d'un automate par élimination d'états (algorithme de

Brzowski-McCluskey, BMC) n'est rien d'autre qu'une application de cette même méthode au système $Lq = \bigcup_r Zq.r.Lr + Fq$.

Choix d'implémentation important : plutôt que d'écrire deux algorithmes séparés, un seul moteur de résolution générique a été implémenté (fonctions `eqsys_solve_for` / `eqsys_solve` dans `equation.c`). Il représente le système sous forme d'une matrice de coefficients (expressions régulières, éventuellement absentes) et élimine les inconnues une à une par la formule générale illustrée figure 3.20 du cours :

$$A[i][j] \leftarrow A[i][j] + A[i][k] \cdot A[k][k]^* \cdot A[k][j] \quad (\text{pour } i, j \neq k)$$

Ce même moteur est utilisé : (1) pour résoudre un système saisi librement par l'utilisateur (option « Systèmes d'équations » du menu), et (2) pour extraire l'expression régulière d'un automate quelconque (fonction `aut_to_regex`), en construisant automatiquement le système $Lq = \bigcup_r Zq.r.Lr + Fq$ puis en ne conservant que la solution de l'inconnue correspondant à l'état initial (ou l'union des solutions s'il y a plusieurs états initiaux). Cette mutualisation illustre concrètement l'unité conceptuelle du cours entre la section 2.3 et la section 3.3.2.

Simplification algébrique

Après chaque union, concaténation ou étoile construite pendant la résolution, une fonction `regex_simplify` applique les propriétés élémentaires du chapitre 2 pour éviter l'explosion de la taille des expressions produites :

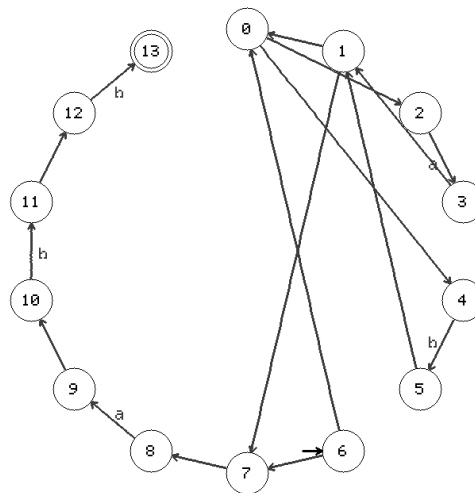
- $X + \emptyset = \emptyset + X = X$ (neutralité de la réunion)
- $X.\emptyset = \emptyset.X = \emptyset$ (absorbant de la concaténation)
- $X.\varepsilon = \varepsilon.X = X$ (neutralité de la concaténation)
- $\emptyset^* = \varepsilon^* = \varepsilon$
- $X^{**} = X^*$ (idempotence de l'étoile)
- $X + X = X$ (idempotence de la réunion)
- $(\varepsilon + X^*) = X^*$

5. Constructions d'un automate à partir d'une expression régulière

5.1 Construction de Thompson (« pure »)

Conformément à la section 3.3.2, chaque sous-expression est traduite en un automate possédant un unique état initial sans transition entrante et un unique état final sans transition sortante (fonctions `th_leaf`, `th_union`, `th_concat`, `th_star` dans `equation.c`, reproduisant les figures 3.13 à 3.18 du cours). La récursion suit directement la structure de l'arbre syntaxique de l'expression.

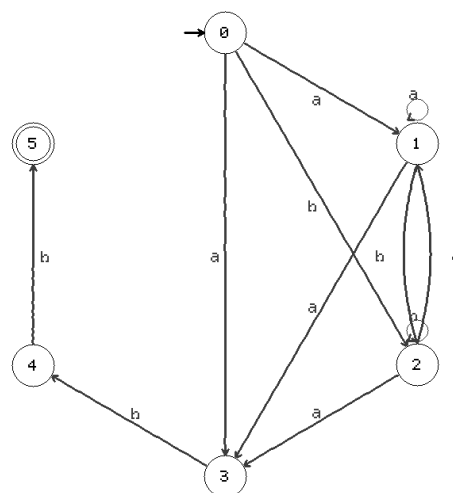
thompson a b abb

Figure 1 — Automate de Thompson pour $(a+b)^*abb$ (généralisé automatiquement).

5.2 Automate de Glushkov (automate des positions)

L'automate de Glushkov associe une position distincte à chaque occurrence de symbole dans l'expression, puis construit un automate déterministe... au sens des positions, sans aucune ϵ -transition. L'algorithme calcule par induction sur l'arbre syntaxique, pour chaque sous-expression : la nullabilité, l'ensemble First (positions pouvant commencer un mot), et l'ensemble Last (positions pouvant le terminer) ; puis, pour chaque nœud de concaténation ou d'étoile, met à jour l'ensemble Follow(p) des positions pouvant suivre la position p. L'automate final possède $k+1$ états pour une expression comportant k occurrences de symboles.

glushkov a b abb

Figure 2 — Automate de Glushkov pour $(a+b)^*abb$ (6 états : l'état 0 plus une position par occurrence de symbole).

Contrairement à Thompson, l'automate de Glushkov ne comporte aucune ε -transition, ce qui en fait un point de départ intéressant pour une analyse lexicale directe.

6. Déterminisation, complétion, émondage

6.1 Construction des sous-ensembles (AFN / ε -AFN $\rightarrow$ AFD)

La fonction `aut_determinize` implémente la construction des sous-ensembles du théorème 2 : chaque état de l'AFD correspond à un sous-ensemble d'états de l'automate de départ, obtenu par fermeture epsilon successive. L'état initial de l'AFD est ε -fermeture(I) ; la transition sortante d'un sous-ensemble T sur un symbole a atteint ε -fermeture($\bigcup_{q \in T} \delta(q,a)$). Cette construction, générique, fonctionne aussi bien pour un AFN sans ε -transitions (auquel cas la fermeture est triviale) que pour un ε -AFN. Le résultat n'est composé que des états effectivement atteints (parcours en largeur), et non de la totalité des 2^n parties possibles.

afd déterminisation

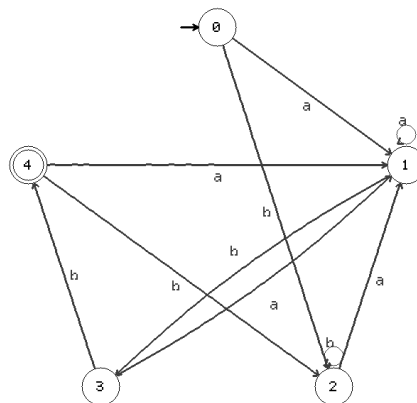


Figure 3 — AFD obtenu par déterminisation de l'automate de Thompson de la figure 1.

6.2 Complétion (AFD $\rightarrow$ AFDC)

La fonction `aut_complete` détecte les couples (état, symbole) sans transition sortante et ajoute, le cas échéant, un unique état puits absorbant (bouclant sur lui-même pour tout symbole), conformément à la construction de la section 3.2.2. Si l'automate est déjà complet, aucun état n'est ajouté.

6.3 États utiles et émondage

Un état est accessible s'il existe un chemin depuis un état initial (fonction `accessible_states`, parcours en point fixe sur les transitions, y compris spontanées) ; co-accessible s'il existe un chemin vers un état final (`coaccessible_states`, parcours sur le graphe inversé) ; utile s'il est à la fois accessible et co-accessible (théorème 1). L'émondage (`aut_trim`) construit l'automate réduit à ses seuls états utiles, renumérotés, conformément à la démonstration du cours.

7. Minimisation et automate canonique

L'algorithme de Moore (section 3.5.3) raffine itérativement une partition des états, en partant de la partition $\Pi_0 = \{\text{finaux, non-finaux}\}$. À chaque itération, deux états de la même classe sont séparés dès qu'ils atteignent, sur un même symbole, deux états de classes différentes ; le raffinement s'arrête lorsque la partition est stable. L'automate résultant (fonction `aut_minimize`) fusionne chaque classe en un unique état représentant.

afd minimal moore

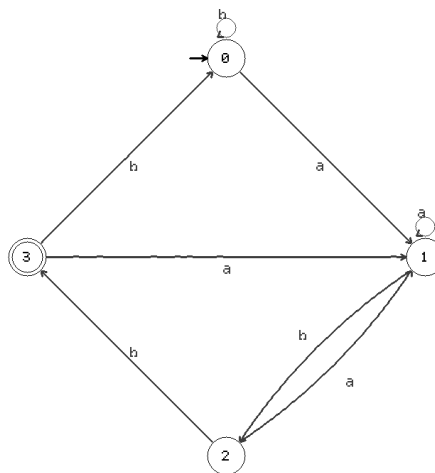


Figure 4 — AFD minimal (Moore) obtenu à partir de la figure 3.

L'automate canonique AL du théorème 16 (Myhill-Nerode) est obtenu en composant l'émondage et la minimisation (fonction `aut_canonical`) : on élimine d'abord les états inutiles pour garantir que le résultat ne contient aucun état « mort » superflu, puis on minimise. Le résultat est le plus petit automate déterministe complet reconnaissant le langage, unique à une renumérotation près, exactement comme démontré dans le cours.

8. Opérations de clôture

Toutes les opérations de clôture démontrées au chapitre 3.3.1 sont implémentées. Le tableau ci-dessous résume, pour chacune, la construction utilisée et le théorème du cours dont elle s'inspire directement.

Opération	Fonction	Construction / théorème du cours
Union (produit)	<code>aut_union_product</code>	Produit d'automates complets, $F = (F1 \times Q2) \cup (Q1 \times F2)$ — Théorème 5
Union (parallèle)	<code>aut_union_parallel</code>	Nouvel état initial + ϵ -transitions vers les deux automates — construction de Thompson, figure 3.16
Intersection	<code>aut_intersection</code>	Produit d'automates complets, $F = F1 \times F2$ — Théorème 6
Complémentation	<code>aut_complement</code>	AFD complet, $F' = Q \setminus F$ — Théorème 4
Différence $A \setminus B$	<code>aut_difference</code>	$A \cap \text{complément}(B)$, via le produit — combinaison des théorèmes 4 et 6
Concaténation	<code>aut_concat</code>	ϵ -transition des finaux de A vers les

Opération	Fonction	Construction / théorème du cours
		initiaux de B — Théorème 8
Étoile de Kleene	aut_star	Nouveaux q_0/q_F , bouclage ϵ sur les finaux — Théorème 9, figure 3.18
Miroir	aut_mirror	Inversion des arcs, échange initiaux $\leftrightarrow$ finaux — Théorème 7

Les opérations binaires (union par produit, intersection, différence) unifient d'abord les alphabets des deux opérandes (fonction `unify_alphabet`), puis déterminisent et complètent chaque automate avant de construire le produit — condition nécessaire à la validité des théorèmes 5 et 6, qui supposent des AFD complets.

union produit

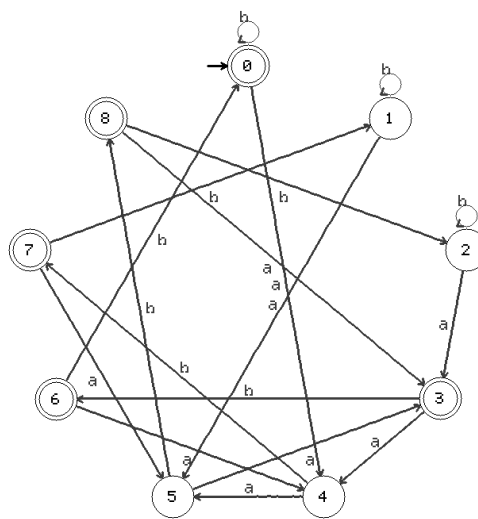


Figure 5 — Union par produit de deux AFD complets reconnaissant respectivement « contient au moins un a » et « contient au moins un b ».

Test d'équivalence

Deux automates sont testés équivalents (fonction `aut_equivalent`) en vérifiant que les deux différences symétriques $L(A) \setminus L(B)$ et $L(B) \setminus L(A)$ sont vides (fonction `aut_is_empty`, qui teste si un état final est accessible), ce qui correspond exactement à la procédure de décidabilité du théorème 14 du cours.

9. États, reconnaissance, énumération, équivalence

La reconnaissance d'un mot (fonction `aut_accepts`) simule l'automate de façon uniforme, qu'il soit déterministe, non-déterministe ou à ϵ -transitions, en manipulant à chaque étape l'ensemble (BitSet) des états courants : après chaque lecture de symbole, l'ensemble est mis à jour par la fonction `move_set` puis refermé par `eps_closure_set`. Le mot est accepté si l'ensemble final intersecte les états finaux. La fonction `menu_simulation` affiche cette trace pas-à-pas pour un usage pédagogique.

L'énumération des mots du langage (aut_enumerate) génère tous les mots jusqu'à une longueur maximale donnée par balayage de l'alphabet et test de reconnaissance ; l' ϵ -fermeture d'un état isolé (option du menu « États ») applique directement l'algorithme 3 du cours (parcours par pile des transitions spontanées).

10. Le moteur graphique PGM

Afin de répondre à l'exigence de visualisation systématique, un mini-moteur graphique a été écrit intégralement en C (module pgm.c), sans bibliothèque graphique externe. Il fournit :

- le tracé de segments par l'algorithme de Bresenham (canvas_line, canvas_thick_line),
- le tracé de cercles par l'algorithme du cercle médian (canvas_circle, canvas_circle_fill),
- le tracé d'arcs courbes quadratiques avec flèche, pour distinguer deux transitions opposées entre les mêmes états (canvas_curved_arc),
- une police bitmap 5×7 couvrant chiffres et lettres, utilisée pour étiqueter états et symboles (canvas_char / canvas_text),
- la sauvegarde au format PGM P5 (canvas_save_pgm).

La fonction aut_render_pgm dispose les états sur un cercle, dessine chaque transition (en regroupant les symboles d'une même paire d'états sur une seule flèche étiquetée, par exemple « a,b »), matérialise l'état initial par une flèche entrante extérieure et les états finaux par un double cercle, puis sauvegarde l'image dans le sous-dossier de results/ correspondant à l'opération effectuée (voir tableau section 11).

11. Interface interactive

Le programme maintient un « espace de travail » persistant : un automate courant A, un second automate B (pour les opérations binaires) et une expression régulière courante. Chaque transformation (déterminisation, minimisation, clôture...) remplace A par son résultat, ce qui permet d'enchaîner naturellement les opérations dans un ordre logique — par exemple : saisir une expression régulière, construire son automate de Thompson, le déterminer, le minimiser, puis en extraire à nouveau l'expression régulière pour vérifier que l'on retrouve un langage équivalent.

Le menu principal se décompose en onze rubriques :

#	Rubrique	Sous-dossier images
1	Définir / générer l'automate courant A	14_divers
2	Définir une expression régulière courante	—
3	Systèmes d'équations (Arden/Gauss) & extraction de regex	03_regex_extraction
4	Transformations sur A (déterminisation, complétion, émondage, minimisation, canonique, miroir...)	02_afn_afd, 04_afdc, 06_emondage, 07_conversions, 09_minimisation, 11_canonique
5	États de A (accessibles, co-accessibles, utiles, ϵ -fermeture)	05_etats
6	Propriétés et tests (déterminisme,	13_mots

	complétude, mot, énumération)	
7	Opérations binaires (union, intersection, différence, concaténation, équivalence)	12_clotures
8	Clôture unaire (complément, étoile de Kleene)	12_clotures
9	Expression régulière → automate (Thompson, Glushkov) & simplification	08_thompson, 10_glushkov
10	Simulation pas-à-pas d'un mot sur A	13_mots
11	Affichage de A (table + image PGM)	14_divers

Chaque saisie est validée (nombre entier attendu, bornes d'états, symbole appartenant à l'alphabet...) avec un message d'erreur explicite en cas de saisie incorrecte, sans jamais interrompre le programme.

12. Validation et tests

Chaque algorithme a d'abord été validé unitairement pendant le développement à l'aide d'un programme de test dédié (accepté/rejeté attendu sur des mots choisis, comparaison des partitions de Moore avec un calcul manuel, etc.). La validation la plus significative concerne la cohérence globale entre trois algorithmes indépendants : la construction de Thompson (regex → automate), l'extraction d'expression régulière (automate → regex, via le moteur d'Arden/Gauss) et le test d'équivalence de deux automates (via produit et complémentation).

Le test suivant a été exécuté pour huit expressions régulières couvrant tous les opérateurs (union, concaténation, étoile, epsilon) : on construit l'automate de Thompson de l'expression, on en extrait une expression régulière équivalente par élimination d'états, on reconstruit un second automate de Thompson à partir de cette expression extraite, puis on vérifie l'équivalence des deux automates.

Expression de départ	Expression extraite (automate → regex)	Équivalent ?
$(a+b)^*abb$	$(a+aa^*a+(b+aa^*b)(b+aa^*b)^*(a+aa^*a))bb$	OUI
$a^*b^*c^*$	$a^*(b^*c^* + b(b^*c^*))$	OUI
$(a+b)^*$	$(a+b)(a+b)^*+@$	OUI
$ab+ba$	$ab+ba$	OUI
$a(a+b)^*b$	$a(a+b)^*b$	OUI
$(ab)^*$	$@+ab(ab)^*$	OUI
$a+b+c$	$a+b+c$	OUI
$(a+@)b^*$	b^*	OUI

Les huit cas sont déclarés équivalents, ce qui constitue une preuve expérimentale forte de la cohérence mutuelle des quatre algorithmes impliqués (Thompson, détermination implicite au sein du test d'équivalence, élimination d'états / Arden, complémentation). Une erreur dans l'un quelconque de ces algorithmes aurait très probablement fait échouer au moins un des huit cas de test, en particulier ceux combinant plusieurs opérateurs imbriqués.

Par ailleurs, la minimisation de l'exemple 5 (AFD redondant reconnaissant les mots finissant par « a », dans lequel les états 2 et 3 sont syntaxiquement distincts mais reconnaissent le même résidu) a été vérifiée manuellement : l'algorithme de Moore fusionne bien ces deux états dès la première itération de raffinement, produisant un automate minimal à 3 états contre 4 au départ, conforme au calcul effectué à la main.

13. Difficultés rencontrées et solutions apportées

13.1 Terminaison des alphabets construits dynamiquement

Plusieurs fonctions construisent dynamiquement un alphabet (collecte des symboles d'une expression régulière, union de deux alphabets pour une opération binaire) dans un tableau de caractères de taille fixe. Une première version oubliait de terminer ce tableau par le caractère nul avant de le transmettre à `aut_init`, qui détermine la taille de l'alphabet par `strlen()` : le programme lisait alors des octets non initialisés au-delà des symboles réels, produisant un alphabet « fantôme » de plusieurs symboles incohérents. Le bogue a été détecté grâce à l'affichage systématique de la table de transition (alphabet visiblement corrompu à l'exécution) et corrigé en ajoutant explicitement le caractère nul de terminaison à la fin de chaque fonction de construction d'alphabet (`collect_symbols`, `unify_alphabet`).

13.2 Unifier automates déterministes, non-déterministes et à ϵ -transitions

Le choix initial d'écrire trois structures de données distinctes (une par famille d'automate) a rapidement été abandonné au profit d'une structure unique paramétrée par un drapeau `hasEpsilon`, car il aurait fallu tripler la plupart des algorithmes (reconnaissance, affichage, rendu graphique). La contrepartie est une consommation mémoire légèrement supérieure (une colonne de transition inutilisée lorsque `hasEpsilon` vaut 0), jugée négligeable au regard des tailles d'automates manipulées dans le cadre du cours.

13.3 Élimination d'états et systèmes d'équations : un seul moteur

La première implémentation de l'extraction d'expression régulière (méthode de Brzozowski-McCluskey) dupliquait la logique déjà écrite pour la résolution de systèmes d'équations. La relecture du cours a fait apparaître que les deux algorithmes sont mathématiquement identiques (résolution partielle par le lemme d'Arden puis substitution) ; le code a donc été refactorisé pour ne conserver qu'un unique moteur de résolution, ce qui a à la fois réduit la taille du code et supprimé une source potentielle d'incohérence entre les deux fonctionnalités.

13.4 Représentation compacte des ensembles d'états

La construction des sous-ensembles (déterminisation) et le calcul des ϵ -fermetures manipulent intensivement des ensembles d'états. Une première approche par tableaux dynamiques s'est révélée à la fois plus lente et plus source d'erreurs (gestion manuelle des doublons) qu'une représentation par entiers 64 bits utilisés comme fanions binaires (`BitSet`), qui rend triviales les opérations d'union, d'intersection et de comparaison d'ensembles, au prix d'une limite de 60 états par automate — largement suffisante pour les exercices du cours.

13.5 Rendu graphique sans bibliothèque externe

Le format PGM ne permettant que des niveaux de gris et aucune primitive de haut niveau, il a fallu réimplémenter depuis zéro les algorithmes classiques de tracé (Bresenham pour les segments et les cercles) ainsi qu'une police de caractères bitmap, et gérer manuellement le chevauchement de deux transitions opposées entre deux mêmes états en les courbant légèrement (fonction `canvas_curved_arc`) pour éviter qu'elles ne se superposent visuellement.

14. Conclusion

Ce TP a permis de traduire en un outil logiciel complet et cohérent l'ensemble des notions abordées dans le cours INF3421 : des mots et langages aux automates finis, en passant par le lemme d'Arden, les constructions de Thompson et de Glushkov, la déterminisation, la minimisation et les propriétés de clôture. Au-delà de l'implémentation individuelle de chaque algorithme, l'exercice a surtout permis d'en éprouver la cohérence mutuelle : le test d'équivalence automatisé (section 12) constitue une vérification croisée qu'aucune séance de cours seule ne permet d'obtenir aussi directement.

La boîte à outils reste perfectible — en particulier la limite de 60 états et le rendu graphique minimaliste pourraient être levés dans une version future en introduisant une structure de données dynamique et un export vers un format vectoriel — mais elle atteint pleinement son objectif pédagogique : rendre manipulables, visibles et vérifiables des concepts qui, sur le papier, restent souvent abstraits.